

Fase de Análise

Front End

Thierson Couto Rosa

Instituto de Informática
Universidade Federal de Goiás

8 de março de 2026

Componentes de uma linguagem de programação

- A partir da linguagem Algol, As linguagens de programação possuem basicamente três componentes bem definidos:
 - Um componente sintático.
 - Um componente léxico.
 - Um componente semântico.
- As linguagens de programação especificam os dois componentes como linguagens formais: **Linguagens Livres de contexto** e **Linguagens Regulares**, o que permite formalizar e automatizar a construção de analisadores para esses componentes (analisadores sintático e léxico).
- Discutiremos cada um desses componentes a seguir.

O Componente Sintático

- O componente sintático de uma linguagem de programação é geralmente descrito no formato *Backus-Naur Form* - BNF.
- A BNF foi primeiramente utilizada para descrever a sintaxe da linguagem IAL [1], atualmente conhecida como ALGOL 58.
- Exemplo:

```
<Stmt> := "while" "(" <Expr> ")" <Stmt> |  
"if" "(Expr)" "then" <Stmt> "else" <Stmt> |  
...
```

- Na BNF <Símbolo> se refere a um *não-terminal*. ::= é um símbolo que significa *é substituído por* e o restante, exceto |, é a sequência de símbolos que substitui <Símbolo>. O símbolo | denota *outra possibilidade de substituição* para <Símbolo>. "símbolo" corresponde a um símbolo *terminal*.

O que é a BNF?

- Exemplo de BNF para a linguagem C.
- A BNF corresponde a um formalismo estudado em Linguagens Formais. Sabe qual é esse formalismo?

O que é a BNF?

- Exemplo de BNF para a linguagem C.
- A BNF corresponde a um formalismo estudado em Linguagens Formais. Sabe qual é esse formalismo?
- Uma Gramática Livre de Contexto (GLC).

O que é a BNF?

- Exemplo de BNF para a linguagem C.
- A BNF corresponde a um formalismo estudado em Linguagens Formais. Sabe qual é esse formalismo?
- Uma Gramática Livre de Contexto (GLC).

Definição

- Uma gramática livre de contexto $G = (V_N, V_T, P, S)$ é um formalismo composto por um vocabulário de símbolos não terminais V_N , um vocabulário de símbolos terminais V_T , um conjunto de produções P e um símbolo especial $S \in V_N$ denominado símbolo inicial.

Exemplo de GLC

$G = (V_N = \{expression, term, factor\}, V_T = \{i, +, -, /, (,)\}, P, expression)$, onde $P = \{$

$expression \rightarrow expression + term$

$expression \rightarrow expression - term$

$expression \rightarrow term$

$term \rightarrow term * factor$

$term \rightarrow term / factor$

$term \rightarrow factor$

$factor \rightarrow (expression)$

$factor \rightarrow (expression)$

$factor \rightarrow i$

$\}$

Adaptações dos símbolos de uma GLC

- Em Linguagens Formais, os símbolos não terminais de uma GLC são geralmente representados por letras maiúsculas do alfabeto e os símbolos terminais por letras minúsculas.
- No caso de linguagem de programação, os símbolos não terminais correspondem a estruturas sintáticas da linguagem, como comandos, expressões, declarações, etc. Logo é importante que sejam representados por nomes que representam essas estruturas (com no exemplo anterior: `factor`, `expression`, `term`).
- Os terminais correspondem a palavras chaves, nomes de variáveis, operadores e delimitadores (`{`, `}`, `[`, `]`), etc, que compõem efetivamente o texto do programa fonte. Logo, os símbolos terminais podem ser formados por mais de um caractere (exemplo: `while`, `x123`).

Interpretação de LFA de programa e de linguagem de programação

- Dada a GLC $G = (V_N = \{S, A, B\}, V_T = \{a, b\}, P, S)$ onde $P = \{S \rightarrow AB, A \rightarrow aA|\varepsilon, B \rightarrow bB|\varepsilon\}$, temos:
 - a cadeia $w = aab \in L(G)$, por quê?

Interpretação de LFA de programa e de linguagem de programação

- Dada a GLC $G = (V_N = \{S, A, B\}, V_T = \{a, b\}, P, S)$ onde $P = \{S \rightarrow AB, A \rightarrow aA|\varepsilon, B \rightarrow bB|\varepsilon\}$, temos:
 - a cadeia $w = aab \in L(G)$, por quê? Porque $S \xRightarrow{*} w$, pois
 $S \Rightarrow AB \Rightarrow aAB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aab$
 - $L(G) = \{w \in V_T^* |$

Interpretação de LFA de programa e de linguagem de programação

- Dada a GLC $G = (V_N = \{S, A, B\}, V_T = \{a, b\}, P, S)$ onde $P = \{S \rightarrow AB, A \rightarrow aA|\varepsilon, B \rightarrow bB|\varepsilon\}$, temos:
 - a cadeia $w = aab \in L(G)$, por quê? Porque $S \xRightarrow{*} w$, pois $S \Rightarrow AB \Rightarrow aAB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aab$
 - $L(G) = \{w \in V_T^* | S \xRightarrow{*} w\}$.
- Se substituirmos G a cima pela gramática G_P que descreve uma linguagem de programação P :
 - a) O que seria uma cadeia w gerada por G_P ?

Interpretação de LFA de programa e de linguagem de programação

- Dada a GLC $G = (V_N = \{S, A, B\}, V_T = \{a, b\}, P, S)$ onde $P = \{S \rightarrow AB, A \rightarrow aA|\varepsilon, B \rightarrow bB|\varepsilon\}$, temos:
 - a cadeia $w = aab \in L(G)$, por quê? Porque $S \xRightarrow{*} w$, pois $S \Rightarrow AB \Rightarrow aAB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aab$
 - $L(G) = \{w \in V_T^* | S \xRightarrow{*} w\}$.
- Se substituirmos G a cima pela gramática G_P que descreve uma linguagem de programação P :
 - a) O que seria uma cadeia w gerada por G_P ? **Seria um programa sintaticamente correto em P !**

Interpretação de LFA de programa e de linguagem de programação

- Dada a GLC $G = (V_N = \{S, A, B\}, V_T = \{a, b\}, P, S)$ onde $P = \{S \rightarrow AB, A \rightarrow aA|\varepsilon, B \rightarrow bB|\varepsilon\}$, temos:
 - a cadeia $w = aab \in L(G)$, por quê? Porque $S \xRightarrow{*} w$, pois
 $S \Rightarrow AB \Rightarrow aAB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aab$
 - $L(G) = \{w \in V_T^* | S \xRightarrow{*} w\}$.
- Se substituirmos G a cima pela gramática G_P que descreve uma linguagem de programação P :
 - a) O que seria uma cadeia w gerada por G_P ? **Seria um programa sintaticamente correto em P !**
 - b) O que seria $L(G_P)$?

Interpretação de LFA de programa e de linguagem de programação

- Dada a GLC $G = (V_N = \{S, A, B\}, V_T = \{a, b\}, P, S)$ onde $P = \{S \rightarrow AB, A \rightarrow aA|\varepsilon, B \rightarrow bB|\varepsilon\}$, temos:
 - a cadeia $w = aab \in L(G)$, por quê? Porque $S \xRightarrow{*} w$, pois
 $S \Rightarrow AB \Rightarrow aAB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aab$
 - $L(G) = \{w \in V_T^* | S \xRightarrow{*} w\}$.
- Se substituirmos G a cima pela gramática G_P que descreve uma linguagem de programação P :
 - a) O que seria uma cadeia w gerada por G_P ? **Seria um programa sintaticamente correto em P !**
 - b) O que seria $L(G_P)$? **a linguagem formada por todos os programas sintaticamente corretos em P !**

Analizador Sintático

- É o módulo do compilador que verifica se o programa-fonte w é uma cadeia válida na linguagem gerada pela GLC G_P da linguagem de programação P .
- Sob o ponto de vista de LFA, se S é o símbolo inicial de G_P , então o analisador sintático verifica se $S \xRightarrow{*} w$.

- É o módulo do compilador que verifica se o programa-fonte w é uma cadeia válida na linguagem gerada pela GLC G_P da linguagem de programação P .
- Sob o ponto de vista de LFA, se S é o símbolo inicial de G_P , então o analisador sintático verifica se $S \xRightarrow{*} w$.
- Mas isso é o papel de um *reconhecedor* em LFA. Qual é o reconhecedor para linguagens livres de contexto?

- É o módulo do compilador que verifica se o programa-fonte w é uma cadeia válida na linguagem gerada pela GLC G_P da linguagem de programação P .
- Sob o ponto de vista de LFA, se S é o símbolo inicial de G_P , então o analisador sintático verifica se $S \xRightarrow{*} w$.
- Mas isso é o papel de um *reconhecedor* em LFA. Qual é o reconhecedor para linguagens livres de contexto? *Pushdown Automata* - PDA - autômato com pilha.

- É o módulo do compilador que verifica se o programa-fonte w é uma cadeia válida na linguagem gerada pela GLC G_P da linguagem de programação P .
- Sob o ponto de vista de LFA, se S é o símbolo inicial de G_P , então o analisador sintático verifica se $S \xRightarrow{*} w$.
- Mas isso é o papel de um *reconhecedor* em LFA. Qual é o reconhecedor para linguagens livres de contexto? *Pushdown Automata* - PDA - autômato com pilha. Logo, o analisador sintático é na verdade um PDA!
- Na prática, um analisador sintático faz algumas outras tarefas como reportar erros sintáticos de w quando $w \notin L(G_P)$ (exemplo: o clássico "; expected in line xx");
- Em inglês, a expressão *parsing* é utilizada para significar *análise sintática* e um analisador sintático é referido por *parser*.
- **Parser = analisador sintático = PDA**

O Componente Léxico

V_T de linguagens de programação

- Nos livros de LFA, um símbolo de V_T de uma GLC corresponde a uma letra minúscula, sendo portanto de fácil identificação.
- Nas linguagens de programação, os símbolos terminais podem ser formados por apenas um caractere ou por palavras (palavras reservadas) que podem ter misturas de símbolos (identificadores em C, por exemplo).
- Quais são os símbolos terminais da linguagem C?

V_T de linguagens de programação

- Nos livros de LFA, um símbolo de V_T de uma GLC corresponde a uma letra minúscula, sendo portanto de fácil identificação.
- Nas linguagens de programação, os símbolos terminais podem ser formados por apenas um caractere ou por palavras (palavras reservadas) que podem ter misturas de símbolos (identificadores em C, por exemplo).
- Quais são os símbolos terminais da linguagem C? **Todos os operadores, pontuações válidas, delimitadores ([], {}), palavras reservadas, identificadores, constantes.**
- Os símbolos terminais nestas linguagens são chamados de *tokens* ou *átomos*.

V_T de linguagens de programação

- O componente léxico de uma Linguagem de programação corresponde ao conjunto de símbolos terminais, isto é, *tokens* da BNF que a descreve. Ou seja, é o vocabulário de símbolos terminais V_T da BNF.
- As linguagens de programação a partir da Algol passaram a descrever os tokens através de *expressões regulares*. Isto significa que os tokens são formalmente *linguagens regulares*.
- As expressões regulares são muito convenientes para descrever os tokens, pois:
 - São padrões concisos e precisos.
 - É possível gerar automaticamente reconhecedores muito eficientes para elas (**Quais são esses reconhecedores? Como podem ser gerados automaticamente? Porque são eficientes?**).

V_T de linguagens de programação

- A expressão regular para um token unitário (linguagem regular unitária) é o próprio token. Exemplo: *while* é um símbolo terminal e a expressão regular que o descreve é *while*. O mesmo acontece para: *if*, *+*, *-*, *+=*, etc.
- Alguns tokens requerem expressões regulares mais complexas. Ex. um identificador pode ser definido pela expressão regular

V_T de linguagens de programação

- A expressão regular para um token unitário (linguagem regular unitária) é o próprio token. Exemplo: *while* é um símbolo terminal e a expressão regular que o descreve é *while*. O mesmo acontece para: *if*, *+*, *-*, *+=*, etc.
- Alguns tokens requerem expressões regulares mais complexas. Ex. um identificador pode ser definido pela expressão regular $[A - Z a - z]([A - Z a - z]_0 - 9)^*$, onde $[A - Z a - z]$ significa a operação "ou" entre todas as letras.
- Lembre-se que uma expressão regular denota um linguagem regular. No caso de identificador, **a linguagem é obviamente infinita!**.
- Então, V_T de uma GLC para a linguagem C é formado pela união das linguagens regulares denotadas pelas expressões regulares que definem os tokens da linguagem.

Parece que temos um conflito conceitual?

V_T de linguagens de programação

- Pela definição de GLC, V_T é um vocabulário e, portanto, um conjunto finito.
- Mas o conjunto V_T de uma linguagem como C é infinito pois é resultante da união de linguagens regulares, sendo algumas delas infinitas, por exemplo, a linguagem dos identificadores, das constantes inteiras e constantes *float*.
- Portanto, temos mesmo um conflito conceitual teórico.
- Na prática o V_T das linguagens de programação, incluindo C é finito por uma questão de capacidade limitada de representação dos computadores. A especificação da linguagem C, por exemplo define tamanhos máximos para identificadores e constantes, o que torna as linguagens a eles correspondentes finitas.

O que é uma linguagem de programação sob a ótica de LFA?

- A final o que é uma linguagem de programação sob o ponto de vista de Linguagens Formais?

O que é uma linguagem de programação sob a ótica de LFA?

- A final o que é uma linguagem de programação sob o ponto de vista de Linguagens Formais?
- É uma linguagem livre de contexto?

O que é uma linguagem de programação sob a ótica de LFA?

- A final o que é uma linguagem de programação sob o ponto de vista de Linguagens Formais?
- É uma linguagem livre de contexto?
- Mas também é uma linguagem regular?

O que é uma linguagem de programação sob a ótica de LFA?

- A final o que é uma linguagem de programação sob o ponto de vista de Linguagens Formais?
- É uma linguagem livre de contexto?
- Mas também é uma linguagem regular?
- Um pouco das duas?

O que é uma linguagem de programação sob a ótica de LFA?

- A final o que é uma linguagem de programação sob o ponto de vista de Linguagens Formais?
- É uma linguagem livre de contexto?
- Mas também é uma linguagem regular?
- Um pouco das duas?
- Resposta:

O que é uma linguagem de programação sob a ótica de LFA?

- A final o que é uma linguagem de programação sob o ponto de vista de Linguagens Formais?
- É uma linguagem livre de contexto?
- Mas também é uma linguagem regular?
- Um pouco das duas?
- Resposta: É uma linguagem Livre de Contexto cujos símbolos do alfabeto de entrada V_T são cadeias de uma linguagem regular sobre um alfabeto.

O que é uma linguagem de programação sob a ótica de LFA?

- A final o que é uma linguagem de programação sob o ponto de vista de Linguagens Formais?
- É uma linguagem livre de contexto?
- Mas também é uma linguagem regular?
- Um pouco das duas?
- Resposta: **É uma linguagem Livre de Contexto cujos símbolos do alfabeto de entrada V_T são cadeias de uma linguagem regular sobre um alfabeto.**
- Essa linguagem regular que forma V_T é a união de um conjunto de linguagens regulares denotadas por expressões regulares - união de linguagens regulares gera uma nova linguagem regular.
- O alfabeto que forma as linguagens regulares são subconjuntos de algum alfabeto de codificação de caracteres (ASCII, EBCDIC, Unicode, etc).

Linguagem de programação como uma linguagem formal

Uma linguagem de programação é uma linguagem Livre de Contexto cujos símbolos do alfabeto de entrada V_T são cadeias de uma linguagem regular sobre uma codificação de caracteres.

- Importância desta visão de linguagem de programação como sendo uma linguagem formal:
 - Permite automatizar completamente as análises sintática e léxica de programas escritos em uma linguagem de programação.
 - Automatizar significa que os projetistas da linguagem (no caso, eu e vocês :)) precisam especificar apenas a gramática e as expressões regulares para ferramentas que geram os analisadores sintáticos e léxicos.
 - Vocês não precisam implementar os analisadores!

Linguagem de programação como uma linguagem formal

Uma linguagem de programação é uma linguagem Livre de Contexto cujos símbolos do alfabeto de entrada V_T são cadeias de uma linguagem regular sobre uma codificação de caracteres.

- Importância desta visão de linguagem de programação como sendo uma linguagem formal:
 - Permite automatizar completamente as análises sintática e léxica de programas escritos em uma linguagem de programação.
 - Automatizar significa que os projetistas da linguagem (no caso, eu e vocês :)) precisam especificar apenas a gramática e as expressões regulares para ferramentas que geram os analisadores sintáticos e léxicos.
 - Vocês não precisam implementar os analisadores! **Vocês já têm todo o conhecimento para implementar uma ferramenta geradora de analisador léxico! Aprenderam em LFA. Vamos apenas recordá-lo na segunda parte da disciplina.**

Geração automática dos analisadores. Por quê?

Geração automática de analisador sintático.

Há um teorema que permite gerar um PDA a partir de uma GLC G . Mas o PDA é o analisador sintático. Então é possível gerar automaticamente um analisador sintático a partir da gramática GLC(BNF) da linguagem.

Geração automática de analisador léxico

Dada uma expressão regular, o teorema de Thompson demonstra como obter um AFND- ϵ que reconheça cadeias da linguagem regular correspondente. A demonstração do teorema é um algoritmo que gera um AFND- ϵ . O analisador léxico é formado pelo conjunto de autômatos correspondentes às expressões regulares.

Geração automática dos analisadores. Por quê?

Geração automática de analisador sintático.

Há um teorema que permite gerar um PDA a partir de uma GLC G . Mas o PDA é o analisador sintático. Então é possível gerar automaticamente um analisador sintático a partir da gramática GLC(BNF) da linguagem.

Geração automática de analisador léxico

Dada uma expressão regular, o teorema de Thompson demonstra como obter um AFND- ϵ que reconheça cadeias da linguagem regular correspondente. A demonstração do teorema é um algoritmo que gera um AFND- ϵ . O analisador léxico é formado pelo conjunto de autômatos correspondentes à expressões regulares.

Conclusão

Os teoremas de LFA são realmente muito importantes em compilação! Ele são algoritmos que são implementados em ferramentas que geram os códigos dos analisadores léxico e sintático.

O Componente Semântico

Análise semântica

- Há várias regras que o projetista de uma linguagem de programação precisa definir que não se enquadram em uma descrição formal.

Exemplos:

- O identificador de uma variável deve ser declarado antes que possa ser utilizado em uma expressão.
 - Compatibilidade de tipo: se uma variável foi declarada como sendo de um tipo, ela deve receber uma expressão que seja do mesmo tipo (eventualmente o próprio compilador faz uma conversão antes da atribuição).
 - Regras de escopo de identificadores
- Infelizmente não existem mecanismos formais para descrever essas regras e muito menos, forma de automatizar a geração de analisadores para elas.
 - Esse conjunto de regras forma o componente semântico do compilador.

Como interagem os três analisadores

- Vimos que um analisador léxico verifica na entrada a existência de tokens pertencentes ao vocabulário de símbolos terminais V_T . O Analisador sintático precisa desses tokens à medida em que ele simula derivações da gramática. Então, é razoável que o analisador léxico seja um módulo chamado pelo analisador sintático sempre que ele precise consumir o próximo token da entrada.
- Em algumas linguagens de programação é possível implementar a análise semântica simultaneamente com a análise sintática. Porém, a grande maioria dos compiladores primeiro gera uma representação intermediária de um programa sintaticamente correto e a análise semântica é feita sobre essa representação. Nesse caso, a fase de análise tem duas sub-fases: análise léxico-sintática e a análise semântica. O compilador implementado na disciplina terá essas duas sub-fases.

Próximos Passos

- Nas próximas aulas serão apresentadas uma ferramenta geradora de analisadores sintáticos (Bison) e uma ferramenta geradora de analisador léxico (Flex).
- Na primeira etapa da disciplina vocês devem apenas aprender a usar essas ferramentas para conseguirem gerar os analisadores correspondentes para a linguagem G-V1.
- Na segunda etapa, vocês aprenderão a teoria necessária para gerar analisadores sintáticos. Na prática, existem várias restrições que precisam ser impostas a um GLC para descrever linguagens de programação. Na verdade, utiliza-se um sub-conjunto de GLCs. Vamos estudar os conjuntos de GLCs e como gerar analisadores para eles.
- Ainda na segunda etapa da disciplina, recordaremos os conceitos de LFA para implementar analisadores léxicos.

[1] Backus, J. W. (1959). "The syntax and semantics of the proposed international algebraic language of the Zurich ACM-GAMM Conference". Proceedings of the International Conference on Information Processing. UNESCO. pp. 125–132.